

Design rationale

1. Problem

My program analyses chess positions. The user gives a position as a FEN string and gets a report about it

There are two things that can change separately. The first one is the type of report: ScoreReport shows a short score, CoachReport says who is better and gives advice for the current phase of the game. The second one is the engine that evaluates the position: PositionalEngine for the opening, MaterialEngine for the middlegame and an old engine DosChessEngine for the endgame. DosChessEngine is legacy code that I am not allowed to change

If I made a class for every pair (ScoreReportWithMaterial, CoachReportWithDos and so on) I would get $2 \times 3 = 6$ classes, and every new report or engine would add even more

2. Design

Bridge: PositionReport is the abstraction. It keeps a ChessEngine in a field and knows only this interface. ScoreReport and CoachReport are refined abstractions. ChessEngine is the implementor with two methods, name() and evaluate(Position, int depth). It returns an Evaluation (score in centipawns from the white side) and when something goes wrong it throws EngineException or one of its subclasses, InvalidPositionException and EngineTimeoutException. MaterialEngine, PositionalEngine and DosEngineAdapter are the implementations

Adapter: DosEngineAdapter implements ChessEngine and wraps a DosChessEngine object. It converts the position into the old format, calls the old method, converts the answer back and turns error codes into exceptions

Complexity module: I chose dynamic implementor selection. PhaseRoutingEngine is also a ChessEngine. It keeps a map from GamePhase to ChessEngine, and on every call it asks the position for its phase and gives the work to the engine for that phase. The phase is calculated from the input itself (number of pieces and move number), so the client never says which engine to use. Endgames go to the adapted legacy engine automatically

3. Why one pattern is not enough

Bridge alone is not enough because every implementation has to implement ChessEngine, and DosChessEngine does not. I cannot change its code, so without an adapter it could not be used in the bridge

Adapter alone is not enough because the legacy engine would work, but the reports would still be connected to concrete engines. Every new report would have to be written again for every engine

So Bridge splits reports and engines into two separate hierarchies, and Adapter lets the old incompatible class join the engine side without changing it

4. Why DosChessEngine is really incompatible

- the method has another name and signature: think(int plies, char[] board, boolean blackToMove) instead of evaluate(Position position, int depth)
- the parameter order is different: depth comes first, board second
- depth is counted in plies (half-moves), so the adapter multiplies the depth by 2
- the board is a char[64] from a8 to h1 with '.' for empty squares, not a Position object
- the result is a plain int from the side that moves, not an Evaluation from the white side, so the adapter changes the sign when black is to move
- it does not throw exceptions. On error it returns NO_SCORE = -32000 and saves an error code that you read with getLastError()

The adapter translates the error codes like this:

- ERR_ILLEGAL_BOARD -> InvalidPositionException ("position is not legal")
- ERR_NO_KING -> InvalidPositionException (same message as the other engines)
- ERR_OUT_OF_TIME -> EngineTimeoutException
- any other code -> EngineException, the code itself is not shown in the message

The adapter also throws if the error code is set but the score looks normal, so a wrong result never gets to the reports. The reports and App never use DosChessEngine, its constants or its error codes

5. Open/Closed principle

- to add a new report I extend PositionReport and write build(), the engines do not change
- to add a new engine I implement ChessEngine and put it in the map in App.createEngine(), the reports and the router do not change

OpenClosedTest checks both cases: it makes a new report and a new engine inside the test and uses them with the existing classes

6. Limitation

The router uses exactly one engine for each phase and has no fallback. If that engine fails, for example the legacy engine runs out of time at a big depth, the whole report fails, even though another engine could give an answer